NailLab – Gesamtarchitektur & Use-Case-Modell

Zusammenfassung des bisher festgelegten Fachkonzepts, der Systembereiche und ihrer Verbindungen

Grundprinzip

Appointment / Termin ist das zentrale Geschäftsobjekt. Es verbindet Kunde, Studio, Mitarbeiter, Services, gespeichertes Design und Zahlung. Design, Service, Termin und Payment bleiben fachlich getrennt.

1. Gesamtsystem – Überblick

KUNDE → Registrierung/Login → Nail Designer → Design speichern → Studio/Services → Warenkorb → Termin → Zahlung

↓

APPOINTMENT

↓

Mitarbeiter/Admin ← Dashboard / Verfügbarkeit / Bestätigung

↓

Behandlung → Design-Bestätigung → Abschluss → Review / Loyalty / Archiv

TECHNIK: Browser/Vue/Inertia → Laravel Backend/API → relationale Datenbank

■ Stripe (nur Payment) ■ Notifications ■ Storage/Fotos/PDF später

2. Rollen & Verantwortlichkeiten

Customer

Registrieren, einloggen, Designs erstellen/speichern/bearbeiten, gespeicherte Designs auswählen, Services in den Warenkorb legen, Studio und freien Termin auswählen, Termin buchen, bezahlen, Termin abschließen, interne Rezension abgeben und Loyalty-Status nutzen.

Employee

Termine und Kundenvorgänge im Studio bearbeiten, Design über Name/Bestell-/Designnummer aufrufen, Design gemeinsam mit Kunde prüfen, Tablet-/Vor-Ort-Abläufe durchführen. Mitarbeiter lehnen Termine nicht ab.

Admin

Studio verwalten, Services und Preise pflegen, Termine bestätigen/verwalten, Dashboard und Archiv verwenden. Aktuell ist ein Admin zunächst einem Studio zugeordnet; mehrere Studios pro Admin bleiben architektonisch offen.

3. Use-Case-Modell – Gesamt

[CUSTOMER]

■■ Registrieren / Login / Profil

■■ Design erstellen → Design speichern → Design auswählen → Design bearbeiten

■■ Services auswählen → Warenkorb verwalten

■■ Studio auswählen → freien Slot auswählen → Termin buchen

■■ Zahlung durchführen / bei Kurzfristtermin auf Bestätigung warten

■■ Termin stornieren / Termin abschließen

■■ Design vor Ort bestätigen

■■ Review abgeben / Google Review separat

■■ Loyalty-Stempel ansehen

[EMPLOYEE]

■■ Kundentermin öffnen

■■ Design über Nummer/Name aufrufen

■■ Design mit Kunde prüfen

■■ Behandlung durchführen

■■ Vor-Ort-Zahlung / Belegprozess unterstützen

[ADMIN]

■■ Studio verwalten

■■ Services & Preise verwalten

■■ Termine verwalten / kurzfristige Anfragen bestätigen

■■ Verfügbarkeit/Slots festlegen

■■ Designs & Archiv verwalten

■■ Benachrichtigungen/Dashboard überwachen

4. Zentrale Prozessketten

4.1 Design → Termin

Nail Designer → funktionale Konfiguration → Duplikatprüfung → bestehendes Design wiederverwenden ODER neues Design anlegen

↓

Design-ID vorhanden

↓

Service(s) im Warenkorb auswählen

↓

Studio + Termin auswählen

↓

Appointment mit Design verknüpfen

4.2 Termin → Zahlung

Termin ≥ 1 Tag im Voraus + Slot frei → direkte Bestätigung/Zahlung möglich

Termin kurzfristig / am selben Tag → nur freie Slots → Anfrage → Admin-Bestätigung → danach Zahlung

Anfrage blockiert den relevanten Slot, damit keine Doppelbuchung entsteht.

Payment-Status bleibt getrennt vom Appointment-Status.

4.3 Behandlung → Abschluss

Termin bestätigt → Kunde kommt → Mitarbeiter öffnet Design → Kunde bestätigt Design auf Tablet

↓

Behandlung abgeschlossen

↓

Kunde: „Termin abschließen“

↓

Appointment = completed → Review + Loyalty + Archiv

5. Design-Bereich – Detailarchitektur

DESIGN EDITOR

CategoryPanel

■■ FormPanel

■■ ColorPanel

■■ DesignPanel

■■ EffektPanel

■■ VerzierungPanel

NailSelectionPanel

■■ Auswahl einzelner Nägel/Finger

Hand/Nail Preview

■■ aktuelle visuelle Darstellung

Preisberechnung

■■ Basis-Service + funktionale Design-Elemente

Design speichern → funktionale Identität prüfen

Funktionale Identität: Form + Länge + Dekorationen + Steine + weitere funktionale Komponenten

Farbe gehört NICHT zur funktionalen Identität.

→ identische funktionale Konfiguration: vorhandene Design-ID wiederverwenden

→ sonst neue Design-ID

Bearbeiten eines bestehenden Designs → niemals direkt überschreiben

→ neue Konfiguration prüfen → vorhandene ID wiederverwenden oder neue ID anlegen

6. Studio, Mitarbeiter & Verfügbarkeit

ADMIN → STUDIO → EMPLOYEES

■

■■ Arbeitszeiten / Schichten

■■ Urlaub / Krankheit

■■ bereits gebuchte Termine

■■ ggf. Service-Kompetenzen

↓

BOOKABLE SLOTS

↓

CUSTOMER

Aktuell: Admin bestimmt die buchbaren Slots manuell.

Später: automatische Ermittlung aus Kapazität, Arbeitszeiten, Abwesenheiten und bestehenden Terminen.

Ein Kunde darf global keine zwei Termine zur selben Zeit in unterschiedlichen Studios haben.

7. Services & Warenkorb

STUDIO definiert Services + Preise

↓

Customer wählt Service(s)

↓

Service Cart (temporäre Auswahl)

↓

Appointment ← mehrere Services zukünftig möglich

↓

Payment / Receipt

↓

Beleg listet die in Anspruch genommenen Services auf.

Wichtig: Der Warenkorb ist nicht das zentrale Geschäftsobjekt.

Das zentrale Geschäftsobjekt bleibt der Appointment.

Das Design wird separat ausgewählt und am Appointment referenziert.

8. Datenmodell – fachliche Beziehungen

USER / CUSTOMER ■■■■■■■■■■■■■■■■

■ ■

■■■ Designs ■■ Design ■■■■■■■■■■

■ ■

■■■ Cart ■■ CartItems ■■ Service■

■ ■

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

▼

APPOINTMENT

■ ■ ■ ■

■ ■ ■ ■■■ Payment

■ ■ ■■■■■■■■ Employee

■ ■■■■■■■■■■■ Studio

■■■■■■■■■■■■■■■■ Service(s)

■

■■■ Status

■■■ Design

■■■ Review

■■■ Loyalty-relevantes Ergebnis

STUDIO ■■< EMPLOYEES

STUDIO ■■< SERVICES

DESIGN ■■< NAILS / funktionale Komponenten

GUEST CUSTOMER ■■ temporäre Identität; Geschäfts-/Archivbelege müssen davon fachlich getrennt betrachtet werden.

9. Payment-Architektur

Fachliche Regel

Stripe ist ausschließlich für Zahlungen zuständig. Zahlung und Terminstatus sind getrennte Zustände. Für kurzfristige Termine ist fachlich vorgesehen: Zahlungsautorisierung bzw. Reservierung und tatsächliche Belastung erst nach Bestätigung. Für ausreichend früh gebuchte freie Termine ist direkte Zahlung vorgesehen.

Appointment requested/confirmed

↓

Payment record

↓

Stripe

■■ Authorization / später Capture (für kurzfristige Fälle)

■■ direkte Zahlung (für automatisch bestätigte Fälle)

↓

Webhook → Payment Status aktualisieren

10. Benachrichtigungen & Archiv

Appointment event → Notification

■■ Bestätigung

■■ Ablehnung / Alternativvorschlag

■■ Zahlungsaufforderung

■■ Abschluss / Dankeschön / Review-Hinweis

Design + Payment + Receipt + Consent record → Admin-Archiv

Consent für Foto-Nutzung: separates Zustimmungs-Record.

Tablet-Signatur zur Designbestätigung: Signatur selbst wird nicht gespeichert; nur Bestätigungs-/Statusinformation.

Fotos fertiger Nägel: optional und nur bei erteilter Zustimmung.

11. Review, Loyalty & Guest

Review

Genau eine interne Review pro abgeschlossenem Appointment. Sie dient dem Qualitätsmanagement und kann für die Homepage verwendet werden. Eine Google-Review ist davon getrennt.

Loyalty

Visuelle Stempelkarte für eingeloggte Kunden. Ein abgeschlossener Appointment zählt als tatsächlicher Besuch. No-Show zählt nicht als Besuch, kann aber einen separaten Loyalty-Bonus erhalten. Beispielregel: nach 10 Besuchen 20 % Rabatt; Regeln sollen konfigurierbar sein.

Guest Customer

Für Kunden ohne Account wird ein Gastkunden-Datensatz verwendet. Dieser soll nach 2 Wochen gelöscht werden. Die fachliche Trennung zwischen temporärer Gastidentität und dauerhaft benötigten Zahlungs-/Beleg-/Archivdaten muss bei der späteren Umsetzung berücksichtigt werden.

12. Statusmodell

APPOINTMENT: requested → confirmed → completed

■→ rejected

■→ alternative_offered

■→ cancelled

■→ cancelled_late

■→ no_show

PAYMENT: eigener Status, unabhängig vom Appointment-Status.

Die konkrete Payment-State-Machine wird zusammen mit dem Stripe-Authorization/Capture-Flow festgelegt.

13. Technische Gesamtarchitektur

INTERNET / DOMAIN

↓

Vue + Inertia Frontend

↓

Laravel Routes / API

↓

■■■■■■■■■■■ BACKEND ■■■■■■■■■■■

■ Auth / Roles / Policies ■

■ Design ■

■ Services / Cart ■

■ Appointment / Availability ■

■ Payment / Notifications ■

■ Review / Loyalty / Archive ■

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

↓

Relationale Datenbank

■

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

↓↓↓

Stripe Storage CI/CD / Cloud

14. Qualität & Deployment

Developer → git push → GitHub → CI/CD

■■ Code prüfen

■■ Tests ausführen

■■ Frontend Build

■■ Deployment

↓

Cloud / Domain

Kern-Tests: Login, Preisberechnung, Warenkorb, Terminbuchung, Doppelbuchungsschutz und weitere zentrale Geschäftsregeln.

15. Bereiche einzeln – Verantwortungsgrenzen

Bereich	Aufgabe / Grenze
AUTH / USER	Identität, Registrierung, Login, Profil, Rollen customer/employee/admin.
DESIGN	Design Editor, funktionale Design-Konfiguration, Speichern, Wiederverwenden, Bearbeiten als neue Version/ID.
SERVICE	Studio-spezifische Dienstleistungen, Preise und künftig mehrere Services pro Appointment.
CART	Temporäre Auswahl von Services; nicht das zentrale Geschäftsobjekt.
STUDIO	Studio-Stammdaten, Zuordnung von Admin/Mitarbeitern, Services und Studio-Kontext.
EMPLOYEE	Mitarbeiter, Termine, Kundenvorgänge, Designprüfung und Behandlung.
AVAILABILITY	Manuelle buchbare Slots jetzt; automatische Ermittlung später.
APPOINTMENT	Zentrales Objekt: Kunde + Studio + Mitarbeiter + Service(s) + Design + Termin + Status.
PAYMENT	Stripe-Zahlungsprozess und Payment-Status; getrennt vom Appointment.
NOTIFICATION	Ereignisbasierte Kommunikation zwischen System, Kunde und Studio.
ARCHIVE / CONSENT	Belege, Design-/Zahlungsnachweise und separate Foto-Einwilligung.
REVIEW	Eine interne Review pro completed Appointment; Google Review separat.
LOYALTY	Stempel-/Bonuslogik auf Basis tatsächlicher Termine.
GUEST	Temporäre Kundenidentität mit geplanter Löschung nach 2 Wochen.
CI/CD	Tests, Build, Deployment und Cloud-Betrieb.

16. Architekturprinzipien für die weitere Umsetzung

1. Appointment bleibt zentral; andere Bereiche liefern Daten/Status an ihn.
2. Design ≠ Service ≠ Cart ≠ Appointment ≠ Payment.
3. Geschäftsregeln gehören ins Backend; Frontend visualisiert und löst Aktionen aus.
4. Zeit-/Preis-/Loyalty-Regeln möglichst konfigurierbar statt fest im Code.
5. Mehrere Services pro Appointment vorbereiten, ohne die erste UI unnötig zu verkomplizieren.
6. Multi-Studio für Admin später ermöglichen, aber zunächst nur ein Studio pro Admin umsetzen.
7. Automatische Verfügbarkeit als spätere Ausbaustufe; aktuell manuell definierte Slots.
8. Öffentliche Galerie-/Design-Sichtbarkeit noch als fachliche Entscheidung offen lassen.

17. Noch offene Entscheidungen

- Wer darf welche Designs in der Galerie sehen? Studio / eigener Kunde / alle Kunden?
- Exakte Kurzfrist-Schwelle und Stornierungs-/Refund-Regeln.
- Konkretes Stripe Authorization/Capture-Verfahren.
- Genaue Review-Felder und Moderationsprozess.
- Loyalty-Regeln (Stempelzahl, Rabatt, No-Show-Bonus).
- Aufbewahrung von Belegen/Finanzdaten bei Löschung eines Gastkunden.
- Automatische Verfügbarkeitsberechnung und Service-Kompetenzen der Mitarbeiter.

Hinweis: Dieses Dokument bildet die im Projekt bisher festgelegten fachlichen Regeln und die vorhandenen Planungsunterlagen zusammen. Offene Punkte sind bewusst als offen markiert.